# RPyC: summing up key features

- Transparency: transparent access to remote objects and methods via netrefs
- Only exposed classes, methods, or attributes are visible remotely
- Both synchronous and asynchronous calls
- Allows callbacks: server can call functions defined on client
- Concurrency: multiple clients can interact safely with the same server
- Pure Python, no stubs, no IDL, no service registry
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts or retries, no server deduplication

Garantisce access transparency grazie a netrefs, NO LOCATION bisogna conoscere hostname e porta, si concurrency transparency

# RPC in Go

- Go provides built-in support for RPC
  - Package `net/rpc` https://pkg.go.dev/net/rpc
  - Type-safe RPC mechanism

- Main features
  - Provides remote access to exported methods of an object
  - TCP or HTTP as “transport” protocols
  - Supports concurrent client requests

# RPC in Go: defining an RPC service

- Define a type (struct) whose methods will be exposed remotely
- Methods must satisfy the following rules:
  - Method type and method name are exported (start with capital letter)
  - Method has two arguments, both exported or built-in types
    - First argument: input (usually a struct)
    - Second argument: pointer to reply struct for response
  - Method returns an error



# Defining an RPC service: example

```
package arith
import "errors"

type Arith struct{}  // Service struct
type Args struct {A, B int}  // Input arguments
type Result int  // Result type for Multiply method
type Quotient struct {Quo, Rem int}  //Result type for Divide method

func (t *Arith) Multiply(args *Args, res *Result) error {
        *res = args.A * args.B
        return nil
}
func (t *Arith) Divide(args *Args, quo *Quotient) error {
        if args.B == 0 {
                return errors.New("divide by zero")
        }
        quo.Quo = args.A / args.B
        quo.Rem = args.A % args.B
        return nil
}
```

# RPC in Go: server

- On server side
  - Create a TPC (or HTTP) server to receive incoming requests
  - Use **`Register`** (or **`RegisterName`**) to make an object's exported methods visible as RPC procedures and publish them on server

```
func (server *Server) Register(rcvr any) error
func (server *Server) RegisterName(name string, rcvr
any) error
```

  - Takes interface as input: `any` is an alias for `interface{}`, allows any value (tipically a pointer to a struct)
  - `Register` uses the type name as service name and registers methods
  - `RegisterName` allows to specify a custom service name



# RPC in Go: server

  - Use **`Listen`** to create a network listener and announce the server on a specific port

```
func Listen(network, address string) (Listener, error)
```

  - Use **`Accept`** to accept connections on the listener and serve requests

```
func (server *Server) Accept(lis net.Listener)
```

    - Concurrency out of the box: `Accept` spawns a separate goroutine for each request
    - `Accept` blocks; to handle other tasks concurrently in main, call `Accept` in a goroutine
  - Can also use an HTTP handler for RPC messages
    - Use `HandleHTTP` and `http.Serve`
    - See example on course site

# RPC in Go: client

- On client side
  - Use **`Dial`** to connect to RPC server at the specified address and port using the given transport protocol (e.g., `"tcp"`)

```
func Dial(network, address string) (*Client, error)
```

    - Use `DialHTTP` for HTTP connection
  - Use **`Call`** to call synchronous RPC
    - `Call` waits for the remote procedure to complete

```
func (client *Client) Call(serviceMethod string, args
any, reply any) error
```

  - Use **`Go`** to call asynchronous RPC
    - `Go` invokes the call asynchronously and signals completion via `Done` channel in `Call` struct

```
func (client *Client) Go(serviceMethod string, args
any, reply any, done chan *Call) *Call
```



# RPC in Go: client

- On client side
  - Struct **`Call`** represents an active RPC

```
type Call struct {
    ServiceMethod string  // The name of the service and method
                          // to call.
    Args any // The argument to the function (*struct)
    Reply any // The reply from the function (*struct)
    Error error // After completion, the error status.
    Done chan *Call // Receives *Call when Go is complete.
}
```

# RPC in Go example: calculator

- Let's consider a simple RPC calculator with two functions: multiply and divide two integers
- Code available on course site



# RPC in Go: synchronous call

- Need some setup in advance of this, see example
- `Call` makes a blocking RPC call: invokes the remote function, waits for it to complete, and returns its error status

```
// Synchronous call
args := arith.Args{n1, n2}
var mulReply arith.Result
err = client.Call("Arithmetic.Multiply", args, &mulReply)
if err != nil {
   log.Fatal("Error in Arithmetic.Multiply:", err)
}
fmt.Printf("Arithmetic.Multiply: %d*%d=%d", args.A, args.B,
   mulReply)
```

```
func (client *Client) Call(serviceMethod string, args any, reply any)
error
```

# RPC in Go: asynchronous call

- Go uses a channel as parameter to retrieve the RPC reply when the call is complete
- The Done channel will signal when the call is complete by returning the same `Call` object
  - If no channel in input, a new one will be allocated by Go

```
// Asynchronous call
divReply := new(Quotient)
divCall := client.Go("Arithmetic.Divide", args, divReply, nil)
replyCall = <- divCall.Done
// check errors, print, etc.
```

channel

```
func (client *Client) Go(serviceMethod string, args any, reply any,
done chan *Call) *Call
```

- Go internal implementation: see https://go.dev/src/net/rpc/client.go?s=8029:8135 - L284



# Go's RPC semantics

- Client-side: no automatic retries and built-in timeouts
- Client returns an error if no reply is received
  - TCP timeout
  - Server may not have received request
  - Server processed request but a failure occurred before sending reply or a network failure occurred
- Server-side: no automatic deduplication of requests
  - Client request has a unique sequence number but is not used for deduplication on server
  - Sequence number is used on client for request tracking
- Best-effort model
- Developer responsibility to implement retries, timeouts, and server-side deduplication

# RPC in Go: marshaling and unmarshaling

- By default, Go uses `encoding/gob` package for marshaling (encoding) and unmarshaling (decoding) RPC parameters https://pkg.go.dev/encoding/gob
  - When you use `net/rpc`, gob is the default encoding/decoding mechanism used automatically
- gob manages binary streams (gobs) exchanged between Encoder (sender) and Decoder (receiver)
  - Each item in the stream is *self-describing*: item is preceded by a specification of its type; pointers are not transmitted, only the values they point to are
- Basic usage: create an encoder, transmit values, use a decoder to receive and decode values
  - Both RPC client and server must be written in Go
- Use gob explicitly when you need encoding/decoding of Go data outside the automatic RPC mechanism or for persistence



# RPC in Go: marshaling and unmarshaling

- Alternatives to gob for Go RPC

1. `net/rpc/jsonrpc` provides JSON-RPC encoding https://pkg.go.dev/net/rpc/jsonrpc
   - Implements JSON-RPC 1.0 for `rpc` package
   - Provides ClientCodec and ServerCodec
   - JSON: human-readable but less efficient than gob
2. Protocol buffers and gRPC instead of `net/rpc`

# Go RPC: summing up key features

- Transparency: transparent access to remote structs and methods
- Only exposed methods are visible remotely
- Both synchronous and asynchronous calls
- Concurrency: multiple clients can interact with the same server
- No IDL, no service registry
- Go automatically handles marshalling and unmarshalling
- Failure semantics: best effort, requires explicit failure handling
  - No built-in timeouts, retries, server deduplication

Garantisce ACCESS transparency, niente location in quanto server hostname e porta garantito anche concurrency transparency



# Comparing RPC so far

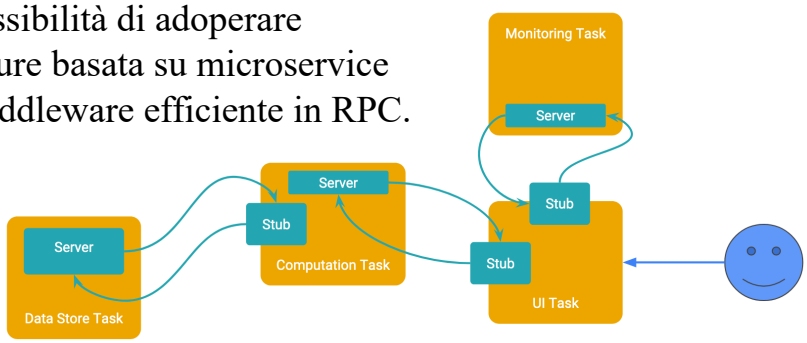
- Let's compare RPC implementations and how they differ in terms of distribution transparency
  - Access transparency
  - Location transparency
  - Concurrency transparency
  - Failure transparency
  - Replication transparency
    - Add server-side load balancer which acts as proxy between RPC clients and replicated RPC servers

tabella completa alla fine

# Motivation for new RPC middleware

- Large-scale distributed applications composed of **microservices**
  - Microservices architecture: build application as a collection of *independent*, *autonomous* (developed, deployed, and scaled independently), *business capability–oriented*, and *loosely coupled* services
  - Multi-language (i.e., polyglot) development
  - Communication mainly structured as RPCs

come visto, data la possibilità di adoperare una software architecture basata su microservice viene introdotto un middleware efficiente in RPC. gRPC






# gRPC

- High-performance, open-source universal RPC framework https://grpc.io/
- Can run in any environment
  - Multi-language, multi-platform support
- Background and adoption
  - Developed by Google and released in 2015, now CNCF project
  - Widely used by companies like Cisco, Discord, Netflix, Reddit, Spotify, Square
  - Etcd, Kubernetes, containerd, GPC use gRPC for communication between components
  - Has become a cornerstone for cloud-native environments



# gRPC

- Main use cases
  - Connect polyglot microservices (request-reply communication style) across and within data centers
    - Supports load balancing, tracing, health checks, authentication
  - Connect devices, mobile apps and browsers to backend services
  - Generate efficient client libraries
- Also core part of many ML systems
  - Gemini, TensorFlow, NVIDIA Triton inference system use it
- And popular for serving models

  https://github.com/tensorflow/serving/blob/master/tensorflow_serving/apis/prediction_service.proto

  https://bentoml.com/blog/3-reasons-for-grpc

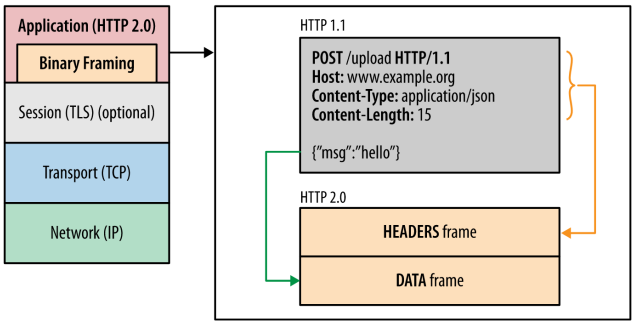


# gRPC: key features

- Transport and communication
  - Uses **HTTP/2** for efficient transport
  - Supports bidirectional streaming, multiplexing, and header compression
- Interface definition and code generation
  - **Protocol Buffers** as IDL
  - Simple service definition, automatic client/server code generation
  - Strict specification prevents errors
- Advanced features
  - Authentication and flow control
  - Blocking or non-blocking client bindings
  - Deadline, timeouts and cancellation
  - Interceptors/middleware: logging, monitoring, retires, metrics

# gRPC transport: HTTP/2

- Basic idea: treat RPCs as references to HTTP objects, leveraging HTTP/2 as underlying transport
- Why HTTP/2?
  - Major revision of HTTP, providing significant performance improvements over HTTP 1.x
  - Enables low-latency, high-throughput communication
- HTTP/2 in a nutshell
  - Binary framing layer: requests and responses are divided into small messages framed in binary, making transmission efficient





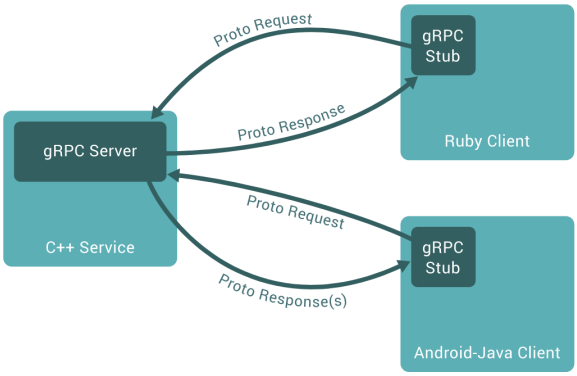

# gRPC transport: HTTP/2

- HTTP/2 in a nutshell
  - From messages to streams
    - Stream: bidirectional flow of bytes within a single connection, may carry one or more messages
    - Message: complete sequence of frames, representing a logical request or response
    - Frame: smallest unit of communication, contains a frame header identifying its stream
  - Bidirectional streaming: supports client-server and server-client streaming simultaneously
  - Request/response multiplexing: multiple requests/responses per client share a single connection, allowing efficient use of TCP connections and avoiding head-of-line blocking
  - HTTP header compression: reduces protocol overhead

See https://hpbn.co/http2/

# gRPC: Protocol buffers

- gRPC uses Protocol buffers (aka *protobuf*) as:
  - IDL for service interface definition and automatic generation of client stubs and abstract server classes
  - Message interchange format: gRPC messages are serialized using protobufs, thus resulting in compact, efficient message payloads
- Design pattern: proxy pattern
  - Client stub acts as a local proxy to the remote service
  - Server skeleton handles incoming RPC calls






# Protocol buffers

- Google's mature open-source mechanism to serialize structured data
- Binary data representation: compact and efficient
- Strongly typed: enforces data safety and prevents erros

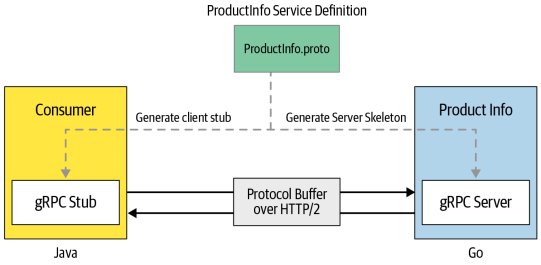
```
syntax = "proto3";

message Person {
  string name = 1;
  int32 id = 2;
  string email = 3;
}
```

- Data types are structured as messages
  - Message: logical record of information containing a series of name-value pairs called fields
  - Each field has a unique number (e.g., `string name = 1`)
    - Field numbers are used in the binary encoding to identify data

# Protocol buffers: example

- ProductInfo service interface



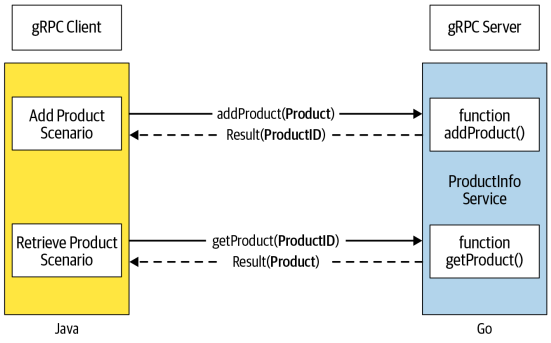

```
// ProductInfo.proto
syntax = "proto3";
package ecommerce;

service ProductInfo {
    rpc addProduct(Product) returns (ProductID);
    rpc getProduct(ProductID) returns (Product);
}

message Product {
    string id = 1;
    string name = 2;
    string description = 3;
}

message ProductID {
    string value = 1;
}
```

Client-server interaction






# gRPC: basic steps

1. Define service and messages: write `.proto` file using protobufs as IDL and describing
   - Service: collection of remote methods
   - Message types exchanged between client and server
2. Generate server and client code
   - Use `protoc` (protocol buffer compiler) in your preferred language(s) from proto definition
   - Go: run `protoc` manually to generate stubs
   - Java: use build tools (e.g., Maven, Gradle)
3. Implement service and client
   - Use language-specific gRPC API (e.g., Go, Java, Python) to write client and server applications
   - Let's consider Go: gRPC-Go https://grpc.io/docs/languages/go

# gRPC: helloworld example in Go

- See https://grpc.io/docs/languages/go/quickstart/ and https://github.com/grpc/grpc-go/tree/master/examples/helloworld/

## 1. Define service (`helloworld.proto` file)

```
package helloworld;

// The greeting service definition.
service Greeter {
  // Sends a greeting
  rpc SayHello (HelloRequest) returns (HelloReply) {}
}

// The request message containing the user's name.
message HelloRequest {
  string name = 1;
}

// The response message containing the greetings
message HelloReply {
  string message = 1;
}
```



# gRPC: helloworld example in Go

## 2. Compile gRPC service definition

```
$ protoc --go_out=. --go_opt=paths=source_relative \
    --go-grpc_out=. --go-grpc_opt=paths=source_relative \
    helloworld/helloworld.proto
```

- Automatically generated files:
  - `helloworld.pb.go`
    - Contains Go structs for protobuf message types: message types have become Go structs
    - Provides methods to populate, serialize, and retrieve request and response data
  - `helloworld_grpc.pb.go`
    - contains Go interfaces (*stub*) for RPC calls: RPC definitions have become Go interfaces
    - Client stub interface to call service methods
    - Server interface to implement service methods
    - Uses message structs from `helloworld.pb.go`

# gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   a. Implement the interface generated from `.proto` service definition: the actual “work”

```
func (s *server) SayHello(ctx context.Context,
   in *pb.HelloRequest) (*pb.HelloReply, error) {
    // Implement service logic here
}
```

   - `ctx`: carries deadlines, cancellation signals and metadata
   - `in`: request message from the client



# gRPC: helloworld example in Go

3. Create gRPC server: composed of two parts
   b. Create and run gRPC server: listen on a network port, create a new gRPC server instance, register the service implementation, pass the listener to the gRPC server

```
lis, err := net.Listen("tcp", port)
if err != nil {
    log.Fatalf("failed to listen: %v", err)
}
s := grpc.NewServer()
pb.RegisterGreeterServer(s, &server{})
s.Serve(lis)
```

   - `Serve` blocks and starts handling requests

# gRPC: helloworld example in Go

4. Create gRPC client
   - Establish a *gRPC channel* (connection) to communicate with server using `Dial`

```
conn, err := grpc.Dial(address, opts…)
```

     - `opts` can include options like `grpc.WithInsecure()`, TLS credentials, etc.
   - Create a client stub to perform RPCs: get it using `pb.NewGreeterClient` provided by `pb` package (generated from `.proto` file)

```
c := pb.NewGreeterClient(conn)
```

   - Call service methods on client stub: send request, receive response

```
r, err := c.SayHello(ctx, &pb.HelloRequest{Name: name})
```

     - Create and populate a request protobuf (`HelloRequest`)
     - Pass a `context` to support deadlines, cancellation or metadata
     - Receive a response protobuf (`HelloReply`)



# gRPC: updating a service

- gRPC simplifies application updates
  - Incremental updates without breaking existing functionalities
- To add a new method
  1. Modify the service definition (`.proto` file): add new methods, update message types
  2. Regenerate client and server stubs
  3. Update server implementation to implement the new method
  4. Update client code to call the new method

# gRPC: updating helloworld example

- Let's update helloworld adding `SayHelloAgain()`
  1. Update the `.proto` file
  2. Regenerate gRPC code using `protoc`
  3. Update server implementation with the new method

```
func (s *server) SayHelloAgain(ctx context.Context, in
   *pb.HelloRequest) (*pb.HelloReply, error) {
    log.Printf("Received: %v", in.GetName())
    return &pb.HelloReply{Message: "Hello again " +
   in.GetName()}, nil
}
```

  4. Update client code calling the new method

```
r, err = c.SayHelloAgain(ctx, &pb.HelloRequest{Name: name})
 if err != nil {
    log.Fatalf("could not greet: %v", err)
 }
 log.Printf("Greeting: %s", r.GetMessage())
```



# gRPC: ProductInfo example

- ProductInfo microservice which manages products and their information
- Client operations:
  - Add new products, retrieve product details

1. Define service in `.proto` file (see slide 122)
2. Implement server and client in Go
3. Implement server and client in Java

Go code: Teams/course website

Java code: https://github.com/grpc-up-and-running/samples/tree/master/ch02

See chapter 2 of gRPC: Up and Running
https://www.oreilly.com/library/view/grpc-up-and/9781492058328/

# gRPC: service method types

- gRPC supports 4 kinds of service methods
  - Defined in `.proto` file
  - Example: route guide
    https://github.com/grpc/grpc-go/tree/master/examples/route_guide

1. Simple (unary) RPC
   - Client sends a single request to server and waits for a single response
   - A normal function call

```
rpc GetFeature(Point) returns (Feature) {}
```

2. Server-streaming RPC
   - Client sends one request to server
   - Server responds with a stream of messages
   - Client reads until the stream ends
   - Message order is guaranteed within an RPC call

```
rpc ListFeatures(Rectangle) returns (stream Feature) {}
```



# gRPC: service method types

3. Client-streaming RPC
   - Client sends a stream of messages to the server
   - Server reads all messages and returns a single response
   - gRPC guarantees message ordering within each RPC call

```
rpc RecordRoute(stream Point) returns (RouteSummary) {}
```

4. Bidirectional streaming RPC
   - Both client and server send streams of messages simultaneously (i.e., full duplex)
   - Each side can read and write independently
   - Server can wait until it has received all client's messages before writing its messages, or server and client can play "ping-pong"
   - gRPC preserves message ordering in each stream

```
rpc RouteChat(stream RouteNote) returns (stream
RouteNote) {}
```

# gRPC: route guide example in Go

- Goal: demonstrates all four types of gRPC communication in Go using a location-based route guide service
- protobufs: service and message definitions in `routeguide/route_guide.proto`
- Server: implements gRPC service methods (`GetFeature`, `ListFeatures`, `RecordRoute`, `RouteChat`)
- Client: calls RPCs using Go gRPC stubs
- Transport: supports both plaintext and TLS-secured connections



# gRPC: distribution transparency

- Access transparency
  - Achieved through the usual proxy pattern
  - Cross-language
- Location transparency
  - Default: DNS name resolver
  - Custom name resolution
    https://grpc.io/docs/guides/custom-name-resolution
  - Service discovery supported via pluggable mechanisms (e.g., Consul, Eureka)
  - Integration with service meshes (e.g., Istio, Consul) that provide service discovery
    - Infrastructure layer for managing service-to-service communication in microservices

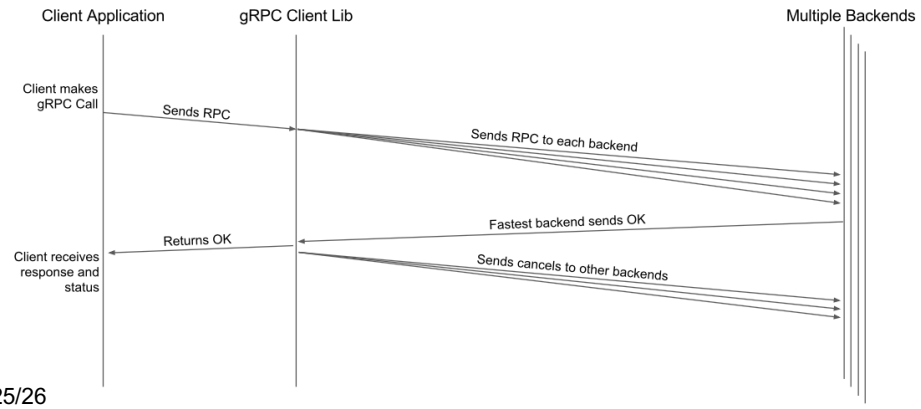
# gRPC: distribution transparency

- Concurrency transparency
  - Provides concurrency-safe APIs, but concurrency management remains application-dependent (no full transparency)
  - gRPC-Go uses goroutines and synchronization mechanisms https://github.com/grpc/grpc-go/blob/master/Documentation/concurrency.md
- Replication transparency
  - Integrates with client-side or external load balancers (e.g., Envoy) to distribute client requests across replicated servers https://grpc.io/docs/guides/custom-load-balancing
  - Replication logic (e.g., keeping replica consistent) must be handled by the application
- Failure transparency
  - At-most-once semantics by default
  - Some failures (e.g., server crashes mid-call) may result in *partial execution*, not rolled back automatically: no exactly-once semantics



# gRPC: distribution transparency

- Failure transparency
  - Retry strategies can be configured for transient failures, including exponential backoff https://grpc.io/docs/guides/retry → retry with backoff pattern
  - Request hedging: send multiple copies of the same request to reduce tail latency https://grpc.io/docs/guides/request-hedging
    - Inspired by: Dean & Barroso, The Tail at Scale, 2013 https://research.google/pubs/the-tail-at-scale/
  - Retries and hedging must be configured, not automatic



# gRPC: additional features

- Security: SSL/TLS support
  - How to use
    - Generate certificates
      - Development: use self-signed certificates (via OpenSSL)
      - Production: use valid, trusted certificates from Certificate Authority
    - Server side: configure gRPC server with SSL/TLS certificates and private key
    - Client side: configure gRPC client to trust server's certificates
    - Mutual TLS (mTLS): for client-server authentication, ensuring both sides are verified
    - Example: https://github.com/grpc/grpc-go/tree/master/examples/features/encryption
- Benchmarking and load testing
  - ghz: modern load testing tool for gRPC https://ghz.sh
  - Test different gRPC call types (e.g., unary vs. streaming)
  - Benchmark with different payload sizes and concurrency levels to simulate real-world traffic



# gRPC: weaknesses

- Limited browser support
  - Cannot directly call a gRPC service from a browser
  - Requires gRPC-Web for communication (but limited features) https://grpc.io/docs/platforms/web/basics/
- Binary protocol
  - protobufs is efficient but not human-readable
  - Requires additional tools like gRPC command-line tool or Wireshark https://wiki.wireshark.org/gRPC
- Load balancing and traffic management
  - No built-in server-side load balancing
  - Service mesh dependency
- Complexity in large-scale systems

# Tabella riassuntiva di RPC

| Tecnologia | Access | Location | Migration | Replication | Failure | Concurrency | Sincronia |
|---|---|---|---|---|---|---|---|
| Sun RPC | Parziale, serve sapere il nome della procedura e i parametri | Parziale, tramite il port mapper serve solo hostname | Non è garantita | No. Ma se introduco un proxy server side posso distribuire il carico | Non è garantita | No, le procedure eseguono in mutua esclusione | No, supporta solo procedure sincrone |
| Java RMI | Si, attraverso pattern proxy, permette di manipolare oggetti remoti | Parziale, tramite RMI registry che mantiene gli oggetti remoti | Non è garantita | No. Ma se introduco un proxy server side posso distribuire il carico | Non è garantita | Parziale, gli oggetti devono essere invocati in contesto thread-safe | No, supporta solo procedure sincrone |
| RPyC | Si, con pattern proxy via netrefs, permette di manipolare oggetti remoti | Non è garantita, serve hostname e porta | Non è garantita | No. Ma se introduco un proxy server side posso distribuire il carico | Non è garantita | Parziale, ci sono diverse configurazioni ma vanno gestiti i lock | Si, supporta procedure asincrone |
| Go RPC (net/rpc) | Parziale, serve sapere il nome della procedura e i parametri | Non è garantita, serve hostname e porta | Non è garantita | No. Ma se introduco un proxy server side posso distribuire il carico | Non è garantita | Parziale, si usano le goroutine ma vanno gestite | Si, supporta procedure asincrone |
| gRPC | Garantita, tramite Protobuf | Non è garantita, serve hostname e porta | Non è garantita | Parziale, tramite un Load balancer esterno | Parziale, permette di fare retry e request hedging se configurato | Parziale, la gestione è application dependent | Si, supporta procedure asincrone |